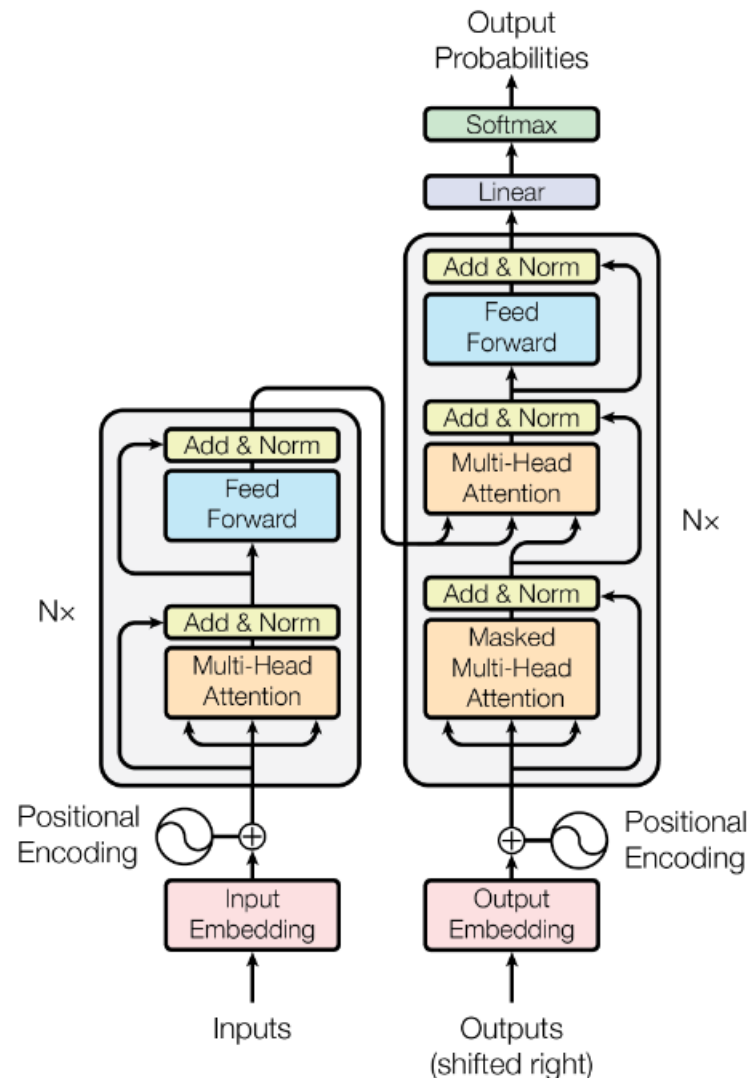


Transformers

K G Atram

Architecture

- Transformer models do not use sequence modelling like RNN
- The Transformer allows for significantly more parallelization of word embeddings
- relying entirely on an attention mechanism



Input to the encoder

Inputs

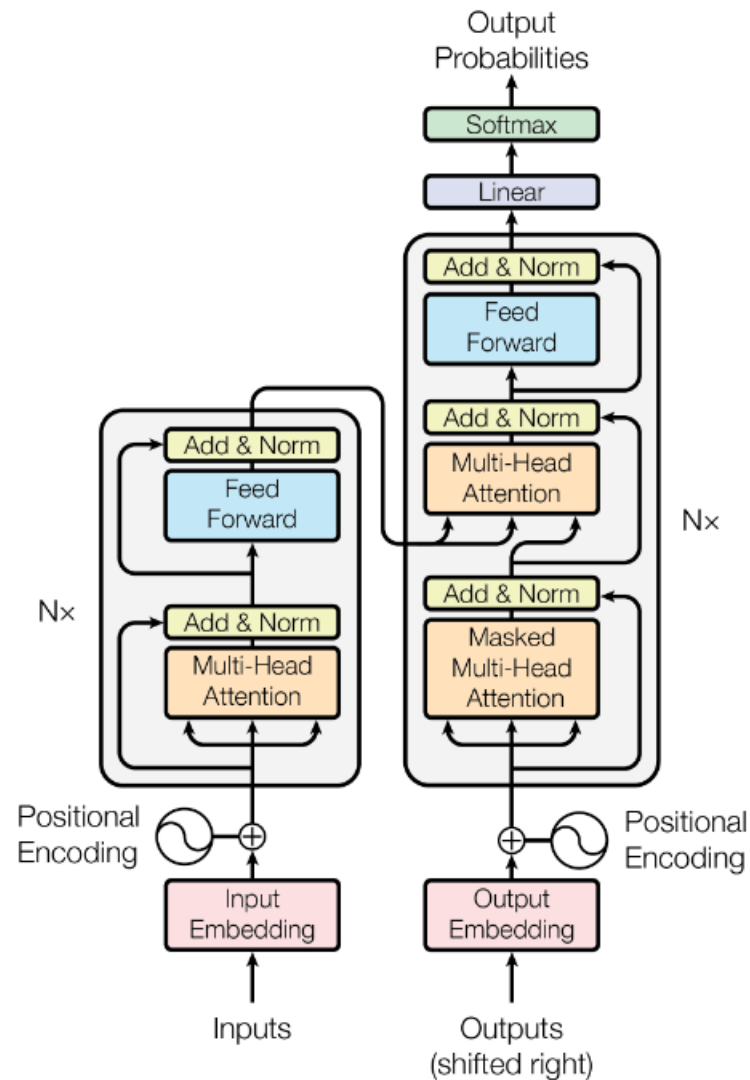
Text

When you play the game of thrones

indices

Vocabulary

a	0
aardvark	1
.	.
play	234
.	.
game	398
.	.
of	607
.	.
oranges	891
.	.
the	987
.	.
thrones	1230
.	.
when	2458
.	.
you	5670



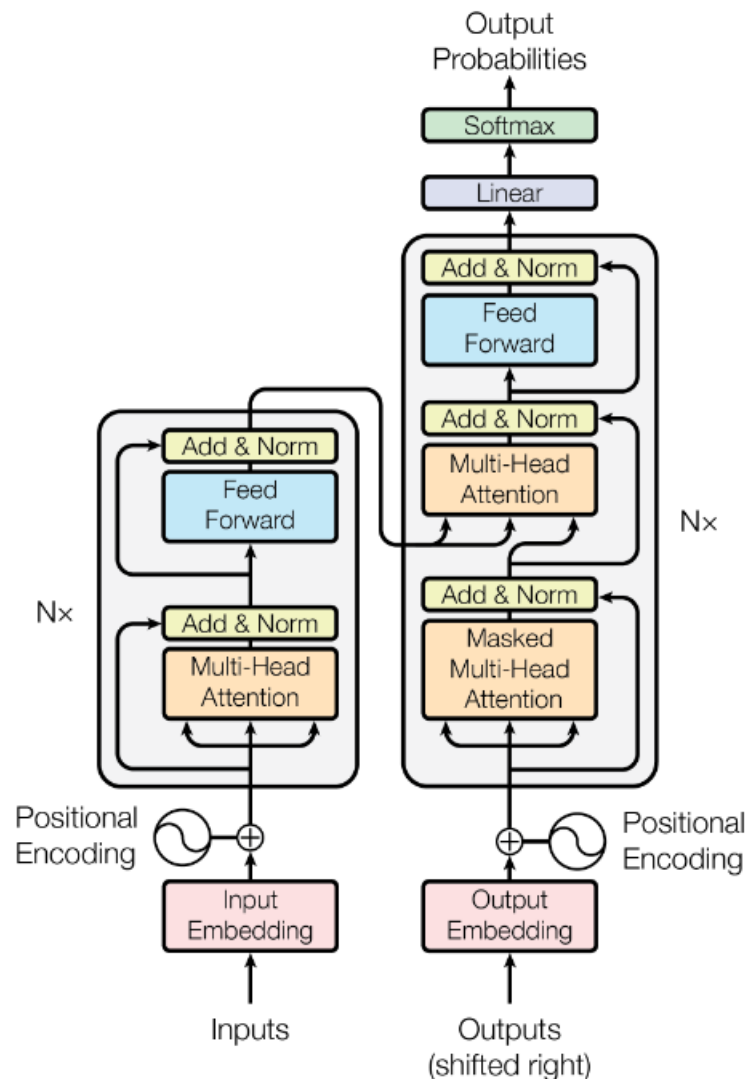
Input to the encoder

Inputs

Text When you play the game of thrones

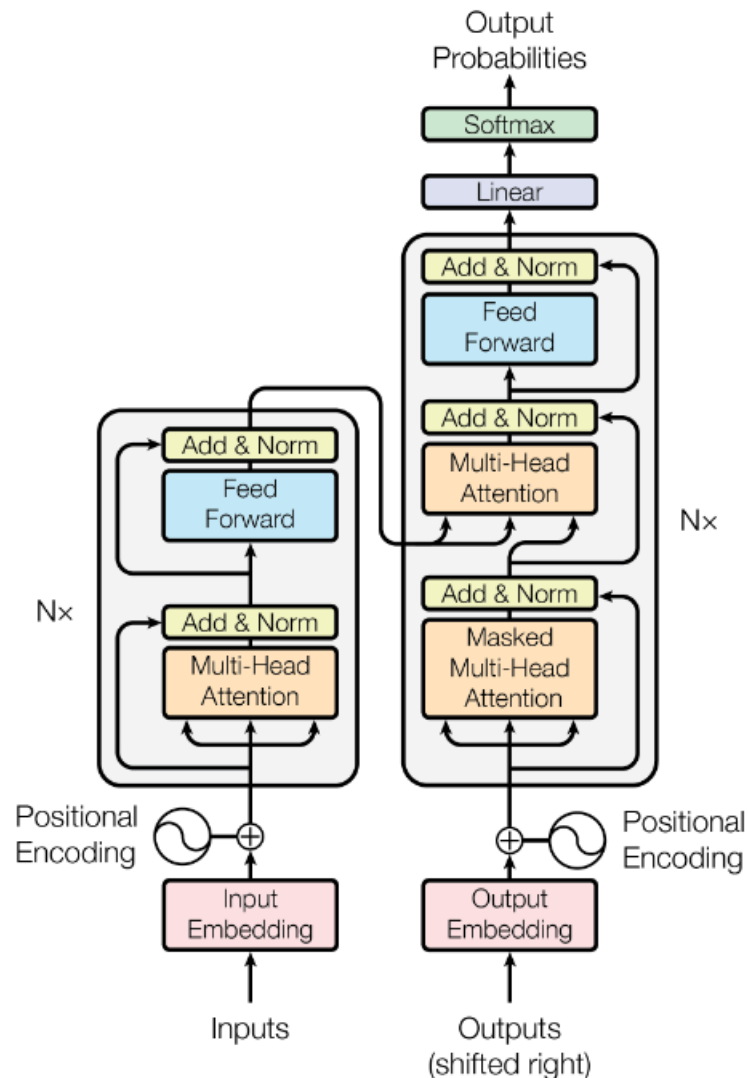
Vocabulary Indices

x_0	x_1	x_2	x_3	x_4	x_5	x_7
2458	5670	234	987	398	607	1230



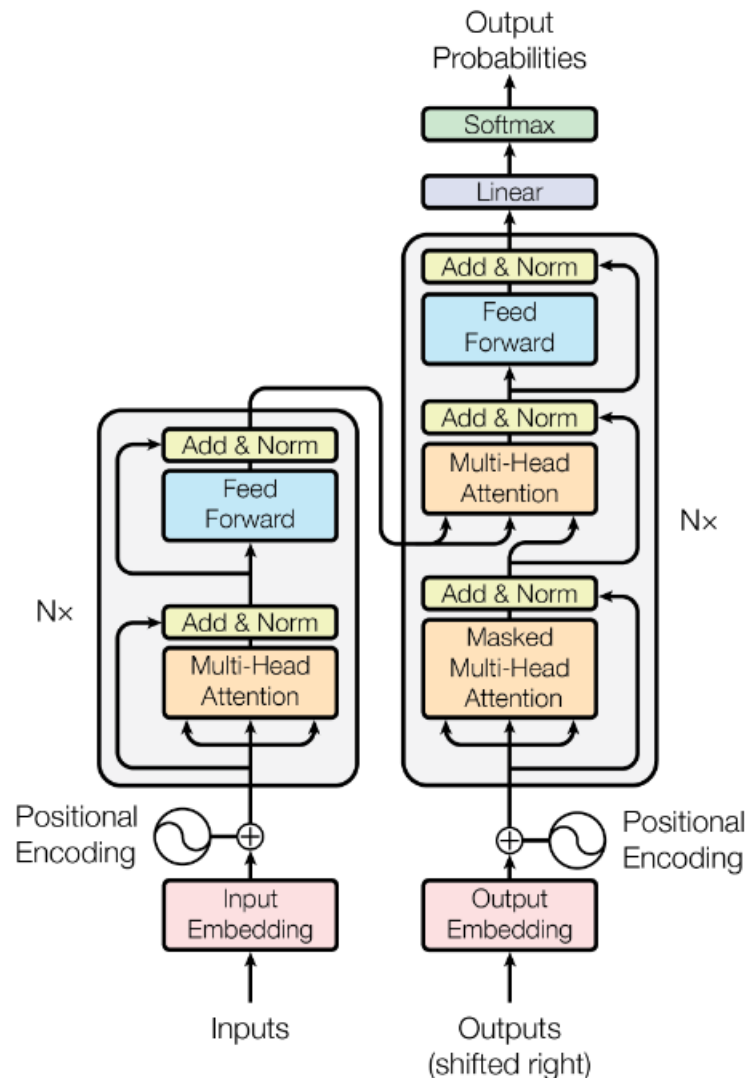
Input Embedding

- Vocab indices are passed to next layer for input embedding.
- Here the indices for each word are converted into vector of 'n' (512) length.



Input Embedding

- Initially these vectors are filled with random numbers.
- Later during training phase, the model update them with values that captures semantic and contextual meaning of word
- The embeddings capture syntactic and semantic relationships useful for the model's downstream tasks.

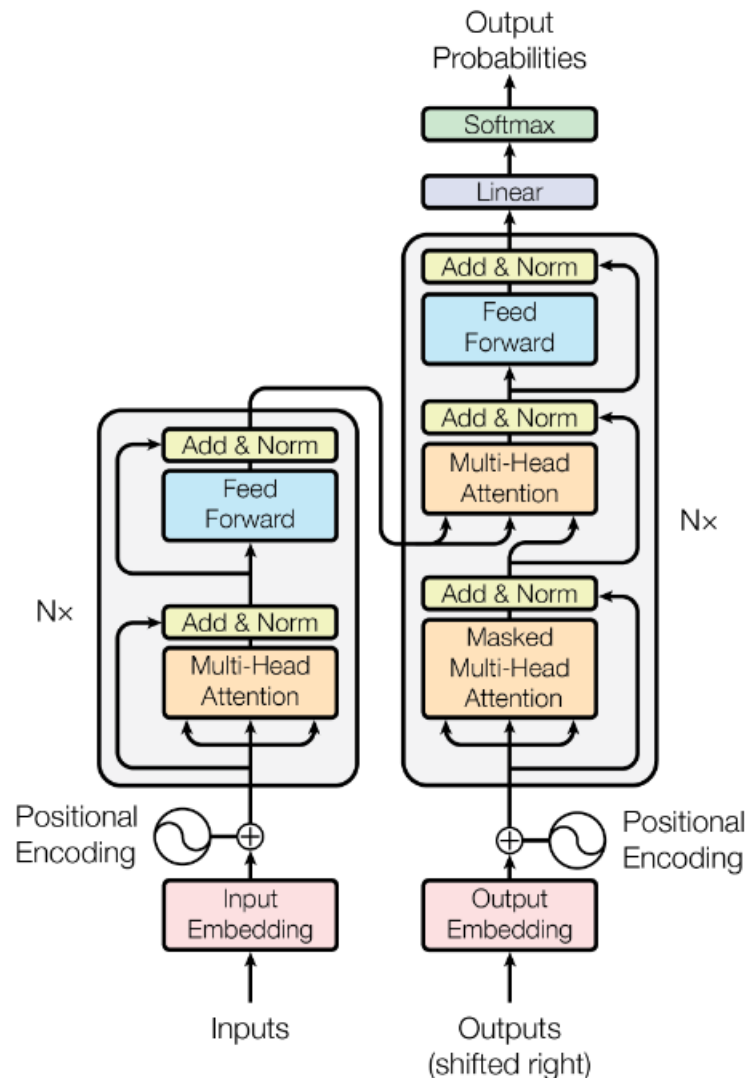


Input Embedding

- Word embedding is vector representation of a word
- The model tries to capture some linguistic features in each dimension about the word such that words used in similar contexts have similar embeddings.

play

0.87	←	Linguistic Feature 1
-0.64	←	Linguistic Feature 2
0.81	←	Linguistic Feature 3
0.26	←	Linguistic Feature 4
-0.35	←	Linguistic Feature 5



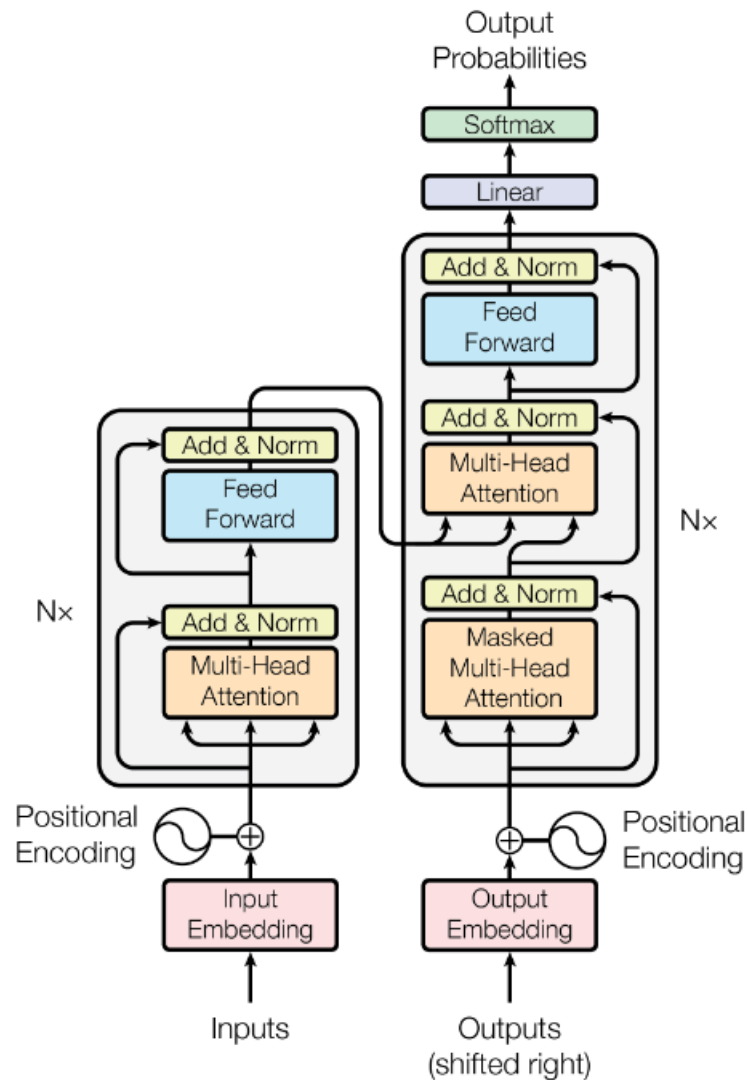
Input Embedding

Inputs Embedding

x_0 x_1 x_2 x_3 x_4 x_5 x_7 Vocab indices

Input Embeddings

e_0	e_1	e_2	e_3	e_4	e_5	e_6
0.87	0.42	0.02	0.38	-0.02	0.45	0.02
-0.64	0.31	0.01	0.16	-0.01	-0.00	-0.21
0.81	0.73	-0.24	0.01	-0.56	0.73	0.13
0.26	0.36	-0.07	0.09	-0.06	0.01	0.01
-0.35	0.99	0.00	0.11	-0.97	-0.02	0.81



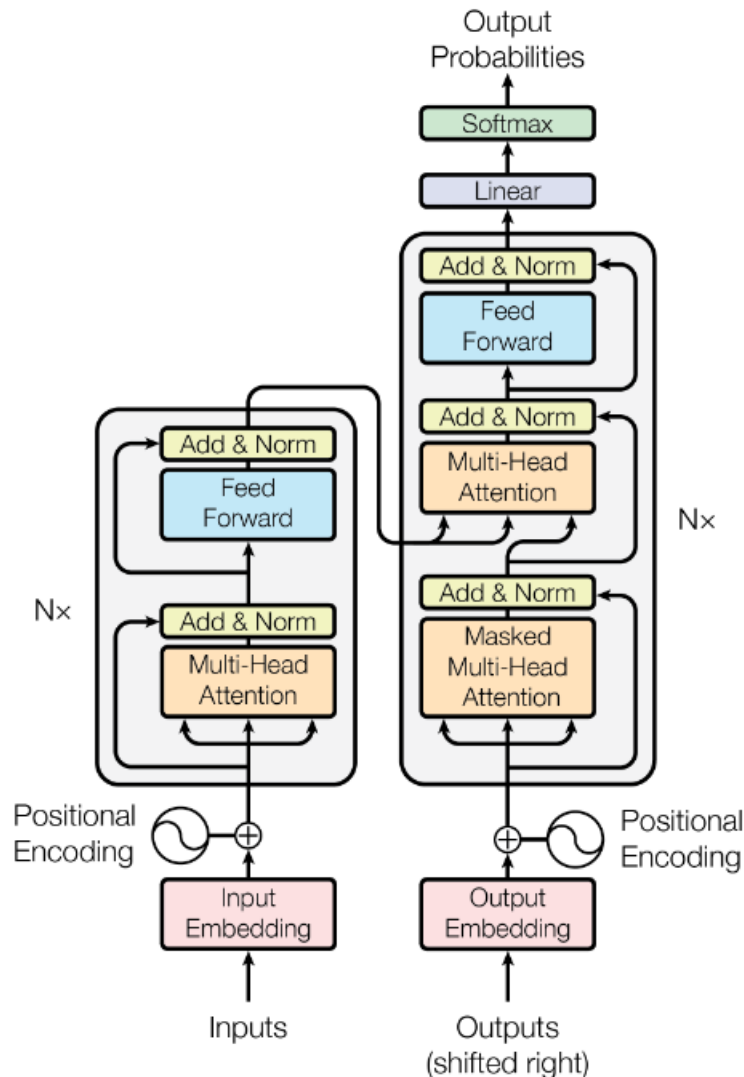
Positional Embedding

- Why do we need positional Embedding?
- Since LSTM process each word sequentially they know the position of each word in the sentence. This is also the reason they are slow.
- On the other hand, transformers process all the words simultaneously therefore they lose position information.

Here is why order matters

Even though she did **not** win the award, she was satisfied.

Even though she did win the award, she was **not** satisfied.



Positional Embedding

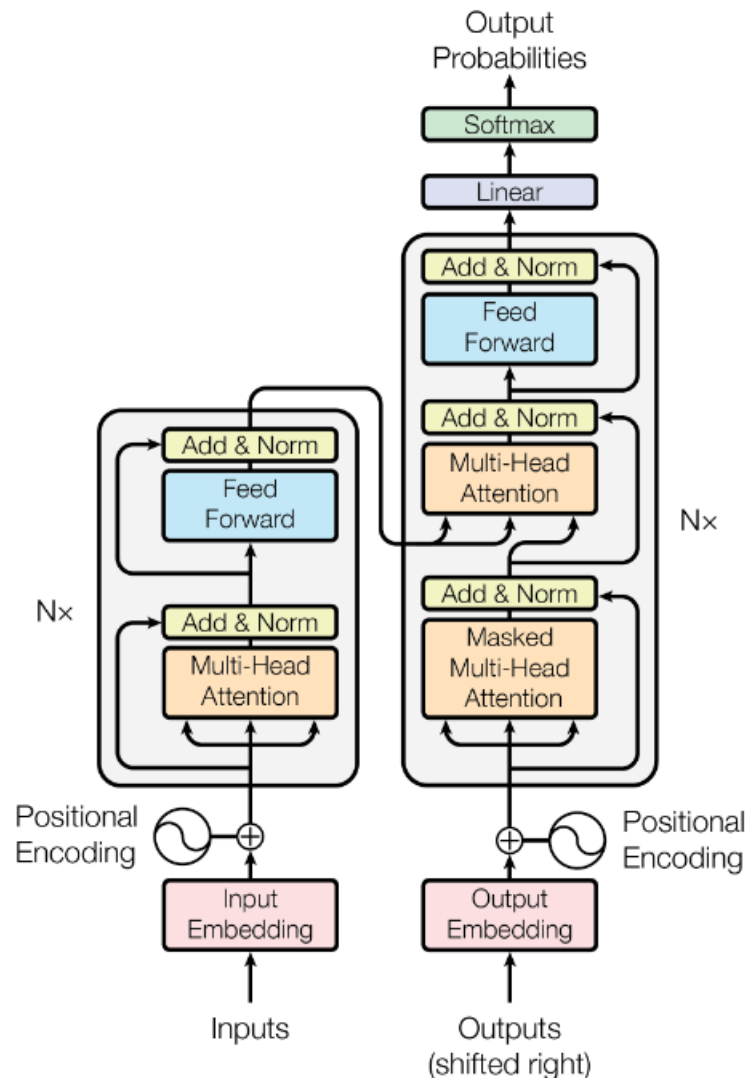
Position Embeddings

When pos = 1		you pos = 2		play pos = 3		the pos = 4	
e_0	p_0	e_1	p_0	e_2	p_0	e_3	p_0
0.42	0.82	0.87	-0.33	0.02	0.27	0.38	-0.78
0.31	0.79	-0.64	-0.25	0.01	0.39	0.16	-0.87
0.73	0.76	0.81	-0.17	-0.24	0.50	0.01	-0.94
0.36	0.74	0.26	-0.09	-0.07	0.60	0.09	-0.98
0.99	0.71	-0.35	-0.02	0.00	0.69	0.00	-1.00

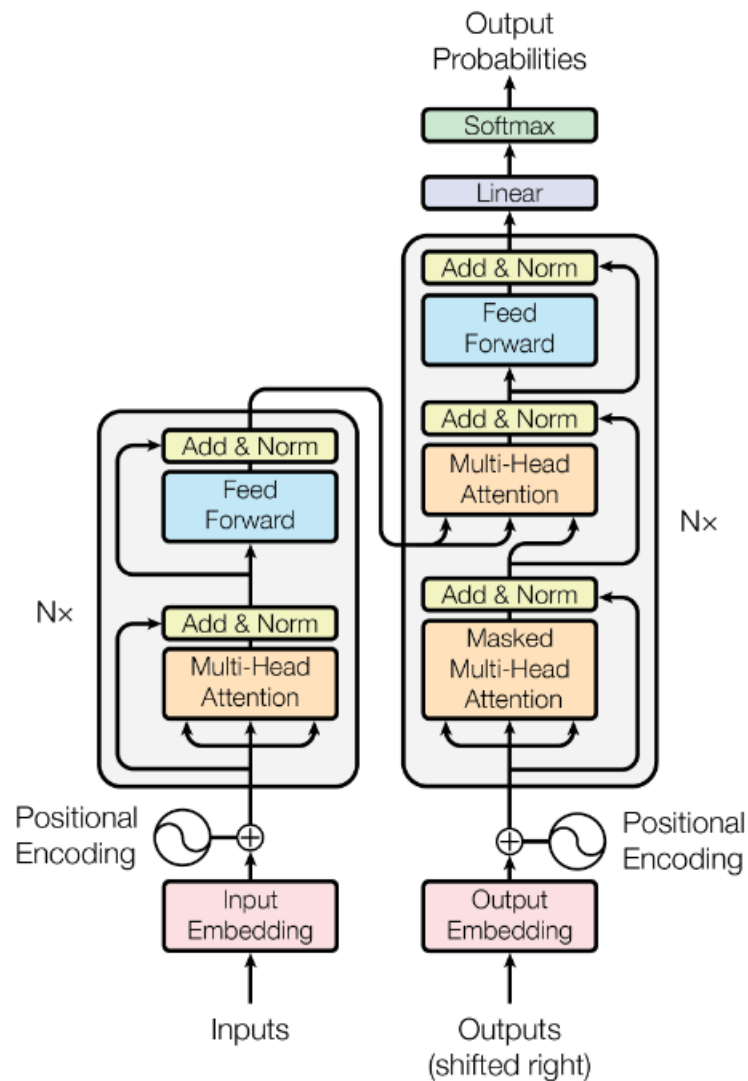
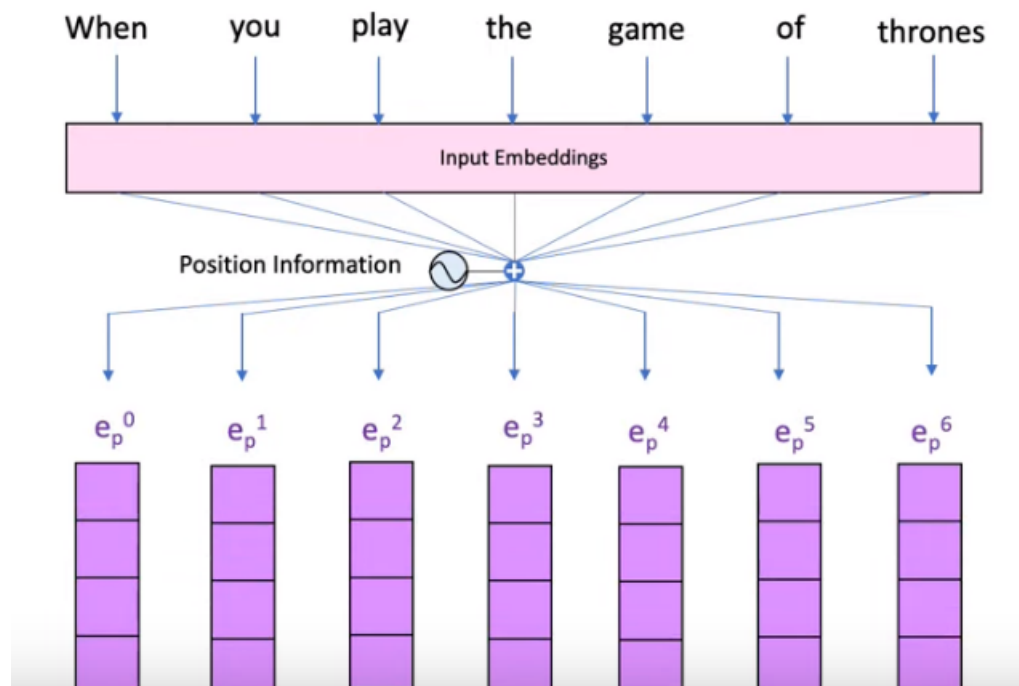
Sin and cos wave freq to capture pos info

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad \text{even } i$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad \text{odd } i$$



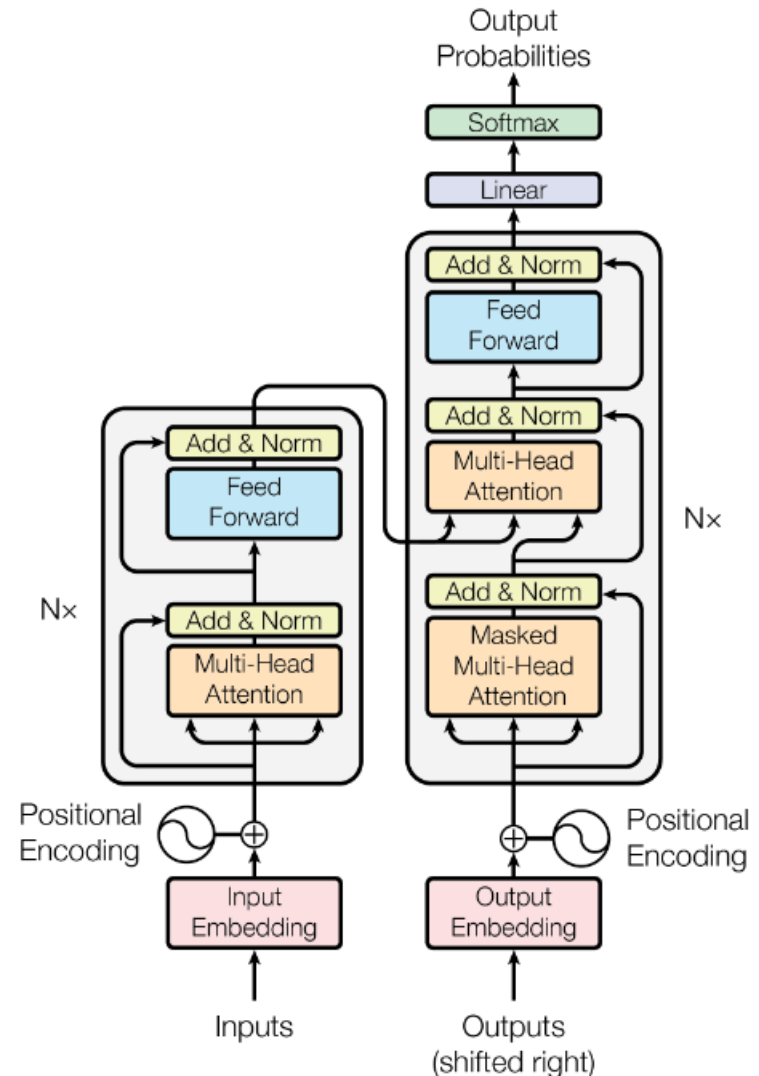
Recap



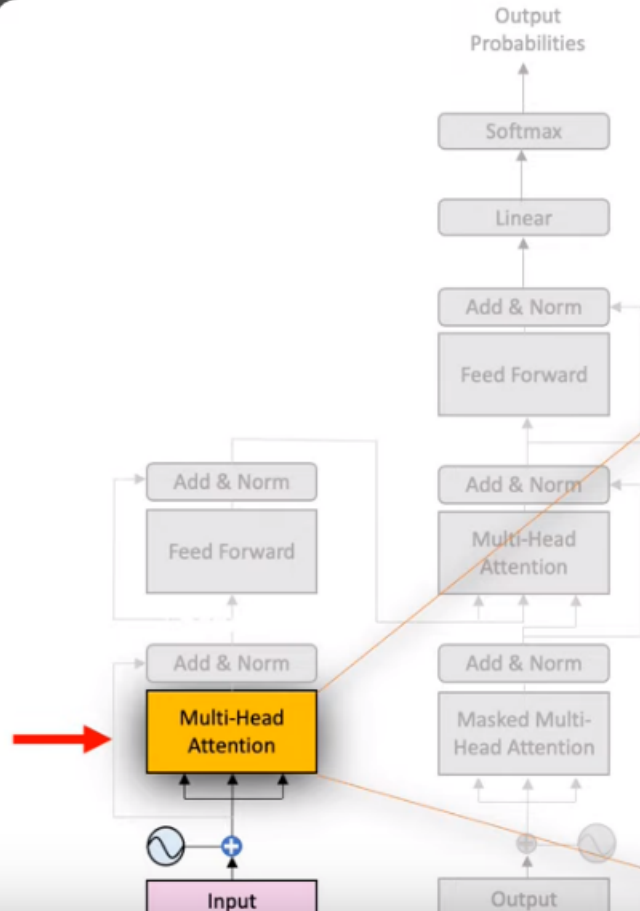
Multi-Head Attention

- Attention mechanism helps the model to focus on important words in given input sentence
- Simple Attention: Selectively focuses on words w.r.t some external query
- He likes Samosas. He like ___ cuisine.
- Self-Attention: also take relationship of words in the same sentence into account

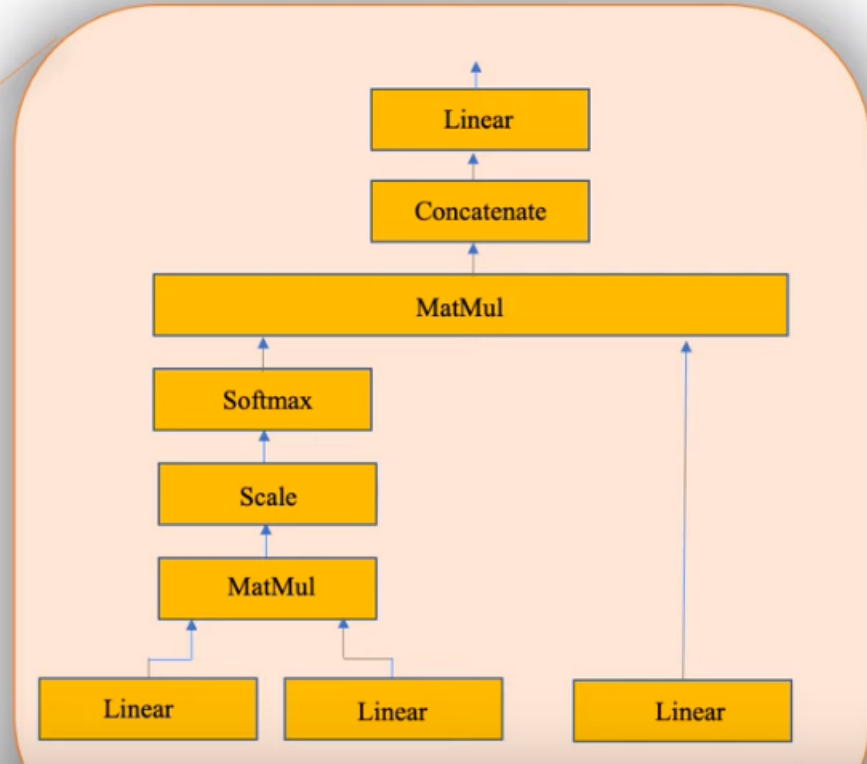
He went to the bank and learned of his empty account, after which he went to a river bank and cried.



Multi-Head Attention



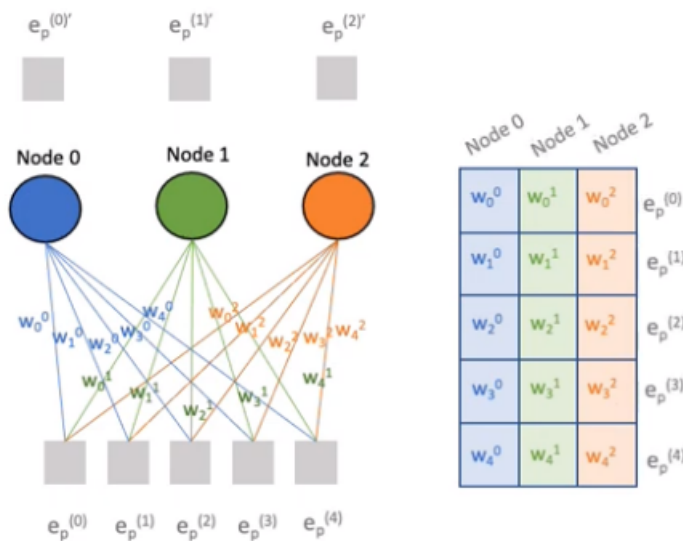
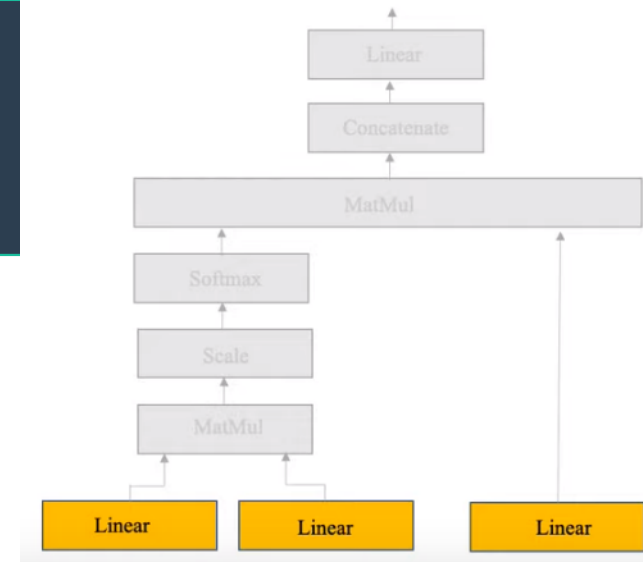
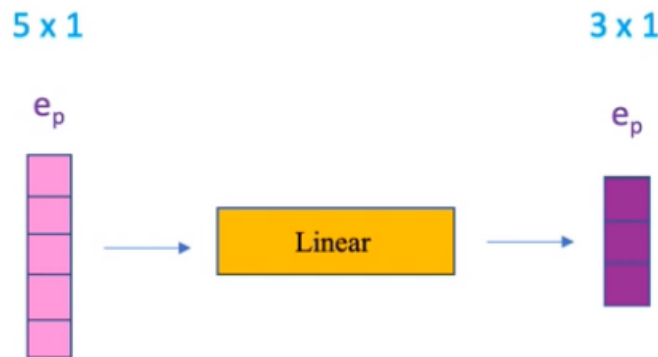
Multi-Head Attention



Multi-Head Attention

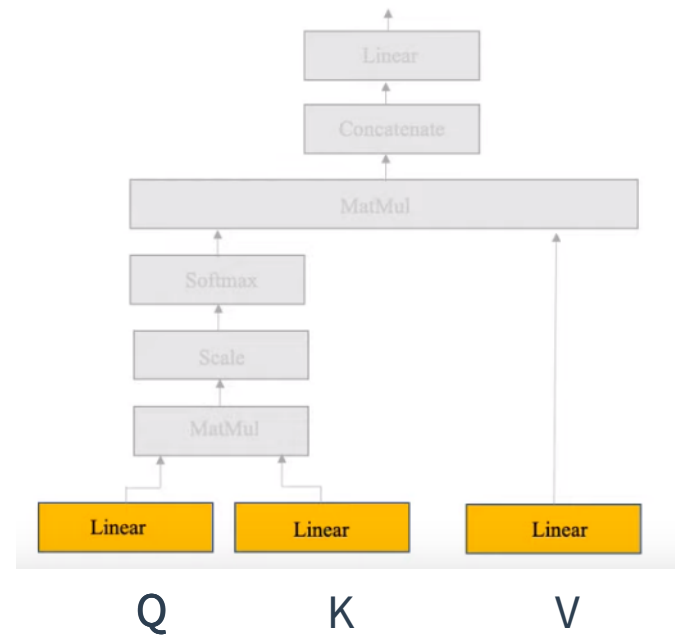
- Linear layer serves two purposes
 - Mapping input to output
 - Changing input vector dimensions

Shrink the dimension of embedding vectors to save on computation cost



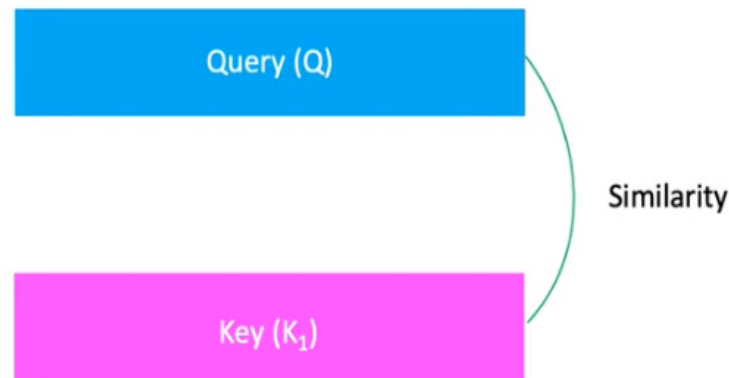
Multi-Head Attention

- Q is Query. Input query
- K is Key. Key of a content
- V is Value. Value associated with keys



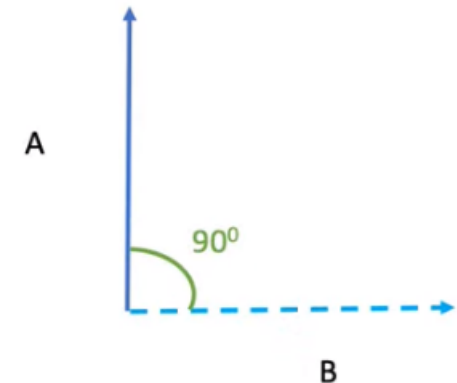
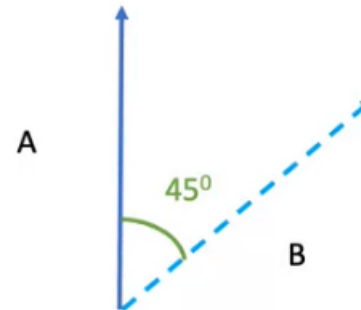
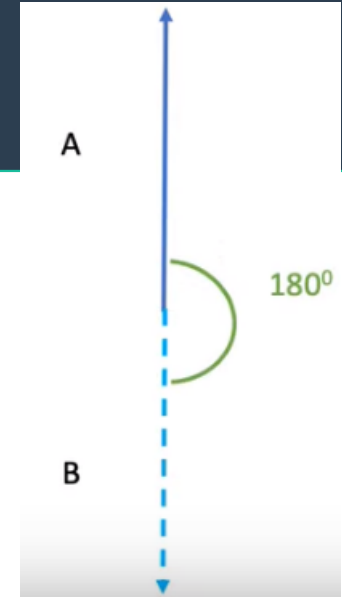
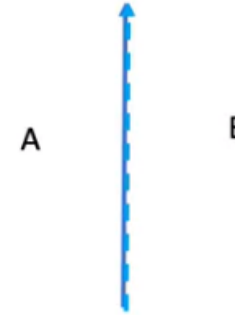
Cosine Similarity

- Similarity can be thought of as a proxy to Attention.
- This is because model returns a value associated with the key which is most similar to the query. In other words, paying attention to similar key.



Cosine Similarity

- Similarity is calculated using cosine similarity
- Cosine similarity varies between -1 (most dissimilar) and +1 (most similar)
- Vector A and B points in same direction.
Angle = 0 , $\cos 0 = 1$
- Angle = 45 , $\cos 45 = 0.7$
- $\cos 90 = 0$
- $\cos 180 = -1$



Cosine Similarity

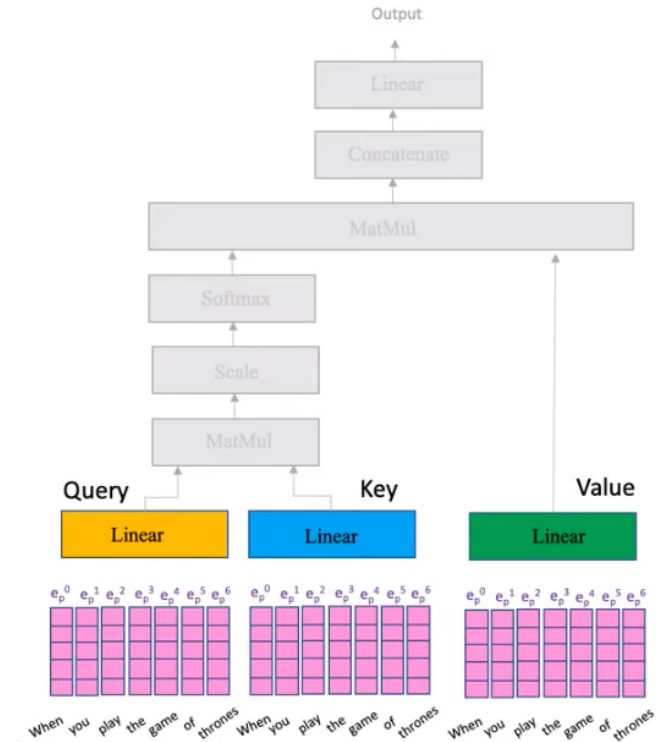
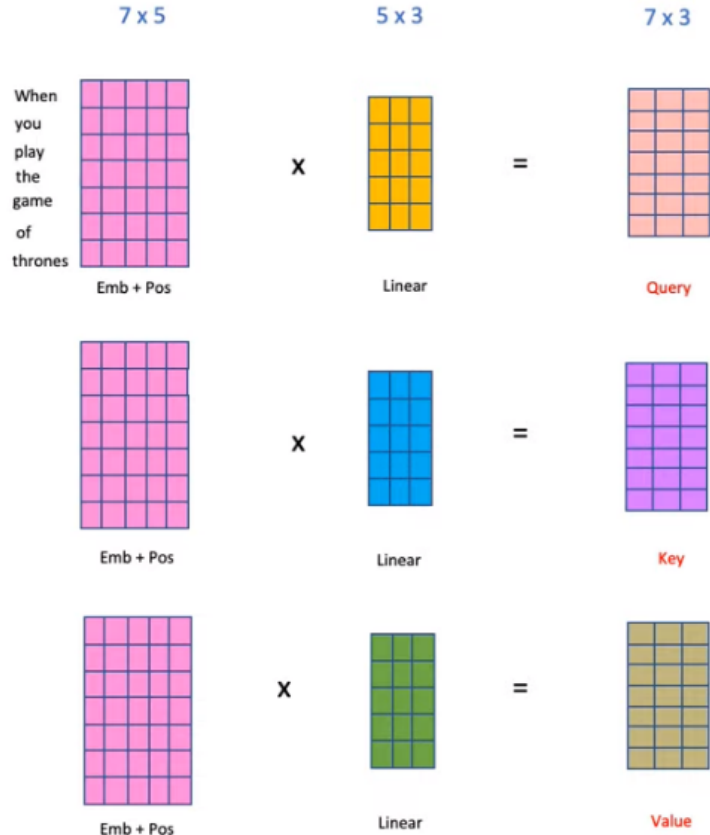
$$\text{Cos}(A, B) = \frac{A \cdot B}{|A| |B|}$$

- $A \cdot B$ is dot product of vector A and B
- $|A|$ is magnitude of vector A
- $|B|$ is magnitude of vector B

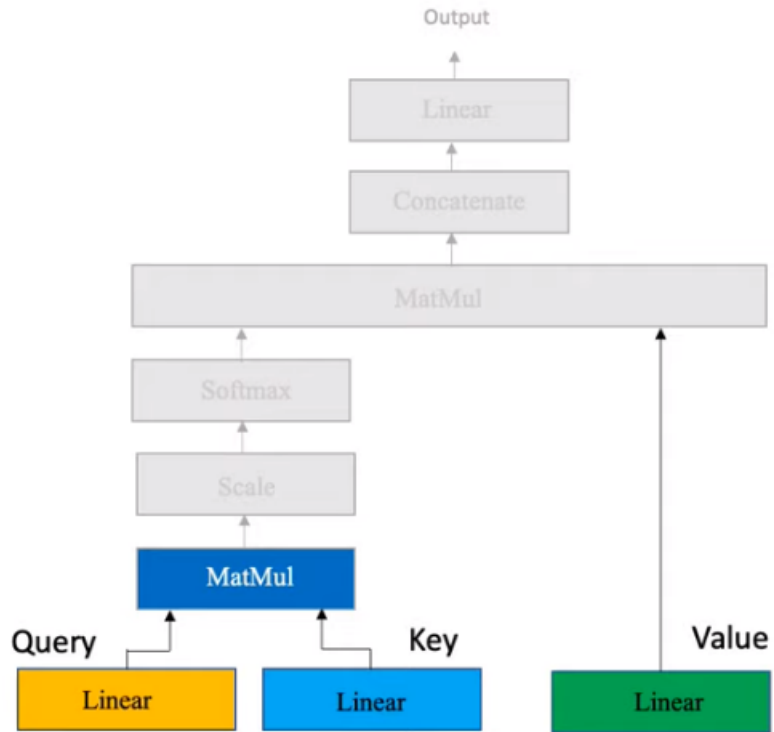
Similarity b/w Query and Key

$$\text{similarity } (Q, K) = \frac{Q \cdot K^T}{\text{scaling}}$$

Multi-Head Attention



Multi-Head Attention



Multi-Head Attention



$$\begin{matrix} 7 \times 3 \\ \text{Query} \end{matrix} * \begin{matrix} 3 \times 7 \\ \text{Key}^T \end{matrix} = \begin{matrix} 7 \times 7 \\ \text{Attention Filter} \end{matrix}$$

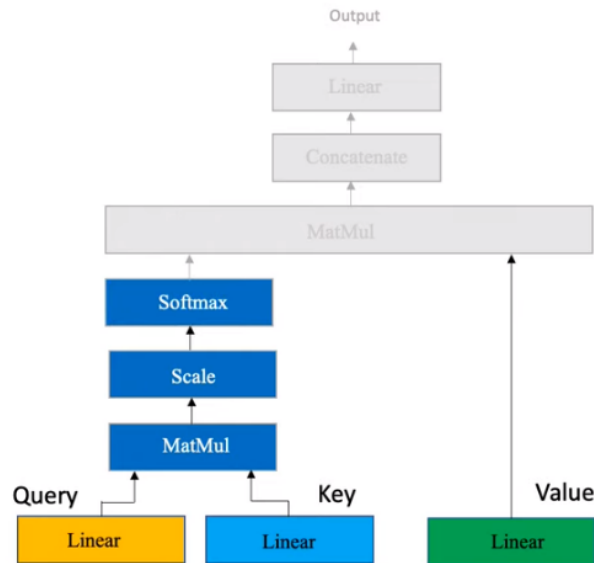
Attention filter

- Attention filter holds attention scores of words w.r.t each other
- $\text{Scale} = \text{Attention score} / \sqrt{d_k}$
- where d_k is dimension of key vector

7 x 7

	When	you	play	the	game	of	thrones
When	89	20	41	10	55	10	59
you	20	90	81	22	70	15	72
play	41	81	95	10	90	30	92
the	10	22	10	92	88	40	89
game	55	70	90	88	98	44	87
of	10	15	30	40	44	85	59
thrones	59	72	92	90	95	59	99

Multi-Head Attention



Softmax

Multi-Head Attention

7 x 7

	When	you	play	the	game	of	thrones
When	33.6	7.6	15.5	3.8	20.8	3.8	22.3
you	7.6	34.0	30.6	8.3	26.5	5.7	27.2
play	15.5	30.6	35.9	3.8	34.0	11.3	34.8
the	3.8	8.3	3.8	34.8	33.3	15.1	33.6
game	20.8	26.5	34.0	33.3	37.0	16.6	35.9
of	3.8	5.7	11.3	15.1	16.6	32.1	22.3
thrones	22.3	27.2	34.8	34.0	35.9	22.3	37.4

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

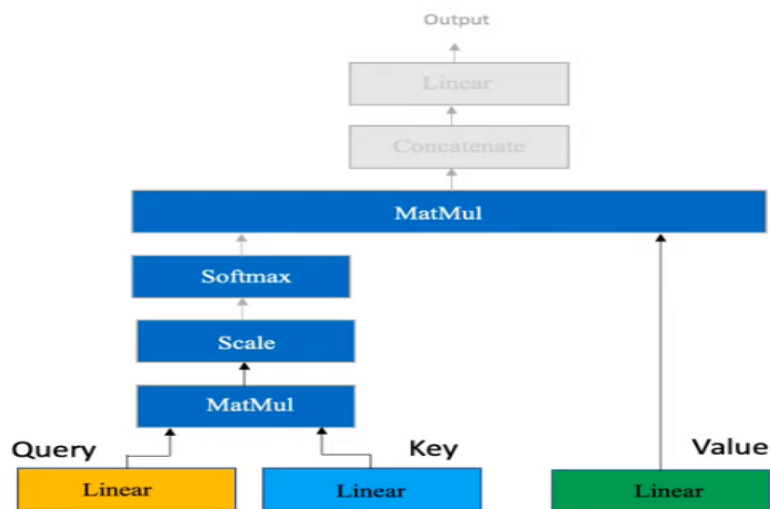
7 x 7

	When	you	play	the	game	of	thrones
When	1.00	0.00	0.00	0.00	0.00	0.00	0.00
you	0.00	0.97	0.03	0.00	0.00	0.00	0.00
play	0.00	0.00	0.68	0.00	0.10	0.00	0.22
the	0.00	0.00	0.00	0.65	0.14	0.00	0.21
game	0.00	0.00	0.03	0.02	0.72	0.00	0.23
of	0.00	0.00	0.00	0.00	0.00	1.00	0.00
thrones	0.00	0.00	0.05	0.03	0.17	0.00	0.75

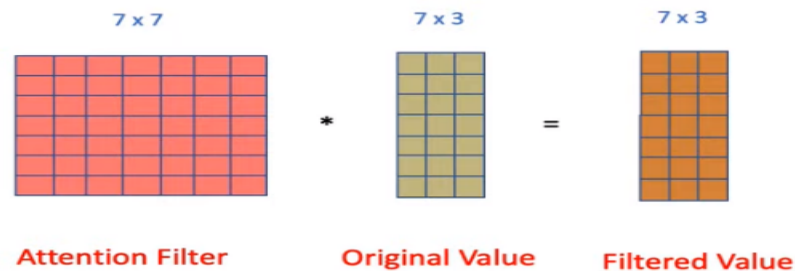
Attention Filter



- When attention filter is multiplied with Value matrix, we get filtered value which assign high focus to the features of more importance



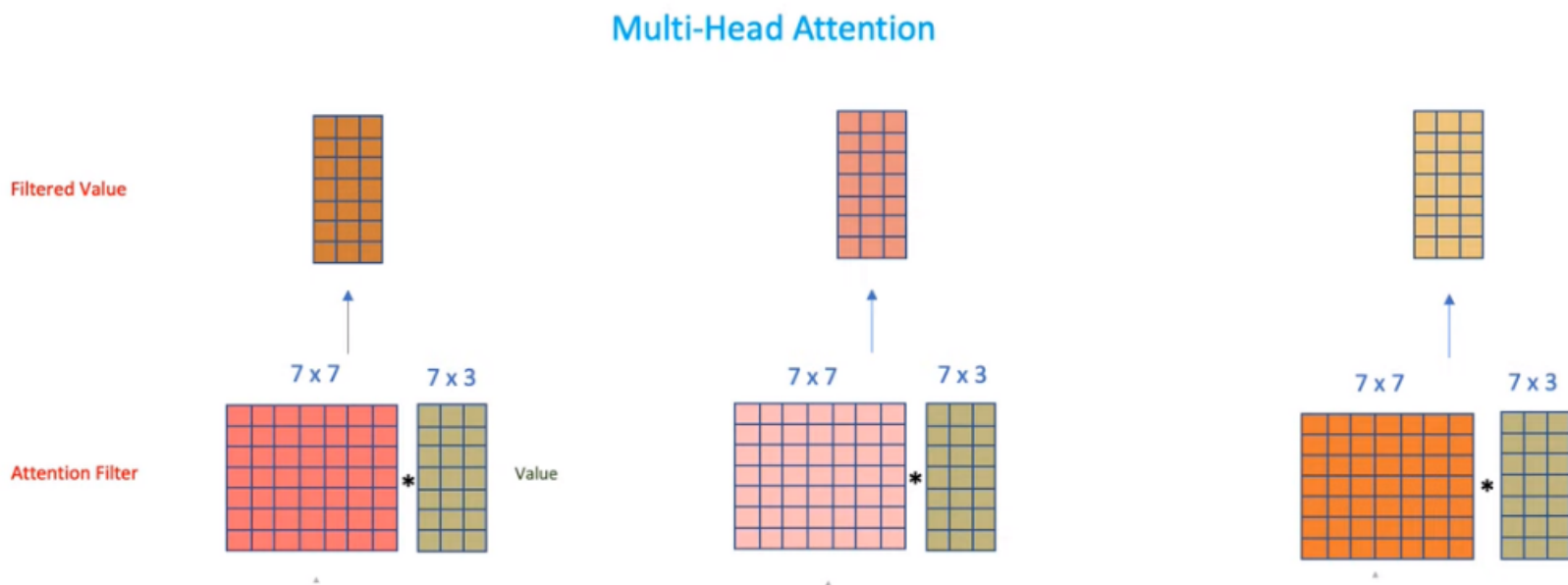
Multi-Head Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

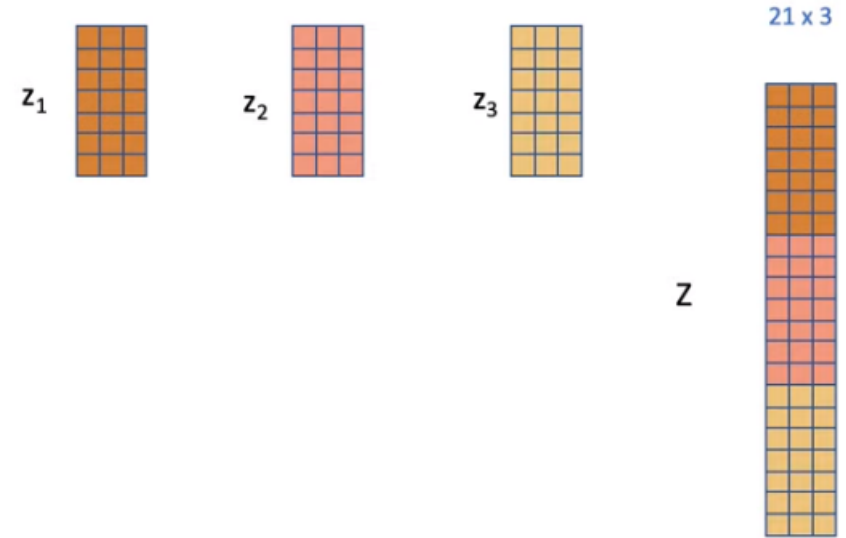
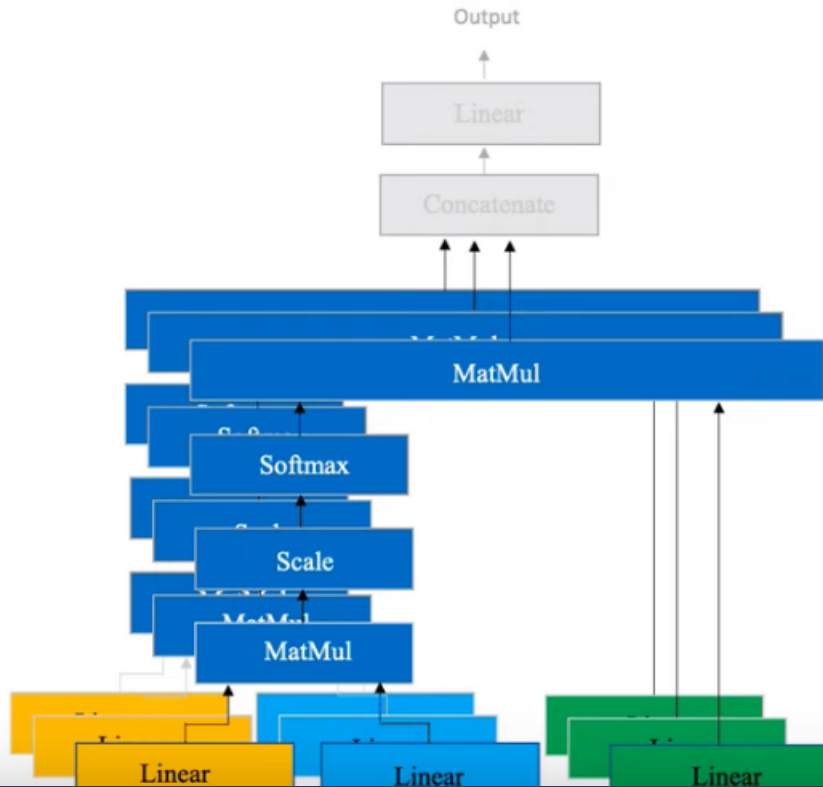
Multi-Head Attention

- Similarly the other attention heads work to learn other linguistic phenomena paying attention to different features.
- In the original paper, authors used 8 attention heads.



Multi-Head Attention

Multi-Head Attention



Multi-Head Attention

